



CMC UNIVERSITY

Aspire to Inspire the Digital World

VIEW & RAZOR

Khoa CNTT&TT, Trường Đại học CMC
Hà Nội, tháng 12 năm 2025

- Razor là một view engine mạnh mẽ được sử dụng trong **ASP.NET Core MVC** để xây dựng giao diện người dùng động. Để học và làm chủ Razor, bạn cần hiểu cả **cơ bản** lẫn **nâng cao**, bao gồm cú pháp, tích hợp với các thành phần của ASP.NET Core, và các tính năng mở rộng.

- **Biểu thức Razor (@)**

<p>Hôm nay là: @DateTime.Now.ToString("dd/MM/yyyy")</p>

- **Khối mã Razor:**

@{

var message = "Chào mừng đến với Razor!";

}

<p>@message</p>

- **Điều kiện**

```
@if (Model.IsLoggedIn) {  
    <p>Chào, @Model.Username!</p>  
} else {  
    <p>Hãy đăng nhập!</p>  
}
```

- **Vòng lặp:**

```
<ul>  
    @foreach (var item in Model.Items) {  
        <li>@item</li>  
    }  
</ul>
```

- **Biến và hàm trong Razor:**

@{

```
string fullName(string firstName, string lastName) => $"{firstName} {lastName}";
```

}

```
<p>@fullName("Nguyen", "Van A")</p>
```

- Hiển thị dữ liệu từ Model:

```
@model Product
```

```
<h1>@Model.Name</h1>
```

```
<p>Giá: @Model.Price</p>
```

- Danh sách dữ liệu:

```
@model IEnumerable<Product>
```

```
<ul>
```

```
    @foreach (var product in Model) {
```

```
        <li>@product.Name - @product.Price</li>
```

```
    }
```

```
</ul>
```

- **Bài tập 1: Hiển thị thông tin cá nhân (Cơ bản)**
- **Mục tiêu:**
 - Làm quen với cú pháp Razor.
 - Sử dụng biểu thức và khối mã Razor.
- **Yêu cầu:**
 - Tạo một View (`Index.cshtml`) hiển thị thông tin cá nhân của sinh viên (Họ tên, ngày sinh, email).
 - Thông tin cần được hiển thị bằng cách sử dụng biến trong Razor.

- **Bài tập 2: Hiển thị danh sách sản phẩm (Cơ bản)**
- **Mục tiêu:**
 - Hiểu cách làm việc với danh sách trong Razor.
 - Sử dụng vòng lặp `foreach` để hiển thị dữ liệu.
- **Yêu cầu:**
 - Tạo một lớp `Product` với các thuộc tính: `Id`, `Name`, `Price`.
 - Trong View, tạo danh sách các sản phẩm và hiển thị dưới dạng bảng HTML.

- ViewData
- ViewBag
- TempData

- **Đặc điểm**
 - Là một **dynamic object** (đối tượng động).
 - Được sử dụng để truyền dữ liệu tạm thời từ **Controller** đến **View**.
 - Không yêu cầu kiểu dữ liệu cụ thể.
- **Phạm vi hoạt động**
 - Chỉ tồn tại trong **request hiện tại** (cùng một HTTP request).
 - Không thể sử dụng giữa các request khác nhau.
- **Ưu điểm và nhược điểm**
 - **Ưu điểm:**
 - Cú pháp đơn giản, dễ sử dụng.
 - Không yêu cầu khai báo kiểu dữ liệu.
 - **Nhược điểm:**
 - Khó kiểm soát và dễ xảy ra lỗi do không hỗ trợ kiểm tra kiểu dữ liệu (type checking).
 - Không phù hợp với dữ liệu phức tạp.

- Trong Controller:

csharp

 Copy code

```
public IActionResult Index()
{
    ViewBag.Message = "Chào mừng đến với trang chủ!";
    ViewBag.Number = 42;
    return View();
}
```

- Trong View:

html

 Copy code

```
<h1>@ViewBag.Message</h1>
<p>Số: @ViewBag.Number</p>
```

- **Đặc điểm**

- Là một **dictionary** kiểu key-value được định nghĩa trong lớp ViewDataDictionary.
- Dữ liệu được lưu dưới dạng **object** và cần ép kiểu khi sử dụng.

- **Phạm vi hoạt động**

- Giống như ViewBag, chỉ tồn tại trong **request hiện tại**.

- **Ưu điểm và nhược điểm**

- **Ưu điểm:**

- Dễ sử dụng và tương thích với các phương pháp cũ hơn.

- **Nhược điểm:**

- Cần ép kiểu khi sử dụng.
 - Không hỗ trợ IntelliSense, dễ gặp lỗi cú pháp.

- Trong Controller:

csharp

 Copy code

```
public IActionResult Index()
{
    ViewData["Message"] = "Chào mừng đến với trang chủ!";
    ViewData["Number"] = 42;
    return View();
}
```

- Trong View:

html

 Copy code

```
<h1>@ViewData["Message"]</h1>
<p>Số: @ViewData["Number"]</p>
```

- **Đặc điểm**
 - Được sử dụng để **lưu trữ dữ liệu tạm thời** giữa các request khác nhau (chẳng hạn: chuyển hướng từ Controller này sang Controller khác).
 - Dựa trên Session để lưu trữ dữ liệu.
- **Phạm vi hoạt động**
 - **Tồn tại trong hai request liên tiếp.**
 - Sau khi được truy cập, dữ liệu sẽ bị xóa trừ khi sử dụng phương thức Keep() hoặc Peek().
- **Ưu điểm và nhược điểm**
 - **Ưu điểm:**
 - Có thể sử dụng để truyền dữ liệu giữa các request.
 - **Nhược điểm:**
 - Dựa trên Session, nên có thể ảnh hưởng đến hiệu suất nếu lưu trữ dữ liệu lớn.
 - Dễ bị mất dữ liệu nếu không sử dụng đúng cách.

- Trong Controller (Lưu dữ liệu):

```
csharp Copy code

public IActionResult Index()
{
    TempData["Message"] = "Dữ liệu từ TempData!";
    return RedirectToAction("About");
}
```

- Trong Controller (Truy cập dữ liệu):

```
csharp Copy code

public IActionResult About()
{
    var message = TempData["Message"]; // Lấy dữ liệu
    return View();
}
```

- Trong View:

```
html Copy code

<p>@TempData["Message"]</p>
```

So sánh

4. So sánh ViewBag, ViewData, và TempData

Đặc điểm	ViewBag	ViewData	TempData
Kiểu dữ liệu	<code>dynamic</code> (động)	<code>key-value</code> (dạng dictionary)	<code>key-value</code> (dựa trên Session)
Phạm vi	Trong request hiện tại	Trong request hiện tại	Qua nhiều request (nếu liên tiếp)
Cần ép kiểu	Không	Có	Có
Hỗ trợ IntelliSense	Có	Không	Không
Ứng dụng chính	Truyền dữ liệu đến View	Truyền dữ liệu đến View	Truyền dữ liệu qua Redirect

- **Bài tập 3: Trang chi tiết sản phẩm (Trung cấp)**
- **Mục tiêu:**
 - Hiểu cách truyền dữ liệu từ **Controller** đến **View** bằng **Model**.
 - Sử dụng cú pháp `@model` trong Razor.

- **Bài tập 4: Trang quản lý sản phẩm (Nâng cao)**
- **Mục tiêu:**
 - Kết hợp layout (_Layout.cshtml) với View.
 - Sử dụng Partial View để hiển thị danh sách sản phẩm.
- **Yêu cầu:**
 - Tạo **_Layout.cshtml** làm giao diện chính, bao gồm:
 - Header với tiêu đề "Quản lý sản phẩm".
 - Footer hiển thị tên ứng dụng và năm hiện tại.
 - Tạo một Partial View (_ProductPartial.cshtml) để hiển thị thông tin sản phẩm.
 - Tạo View chính (Index.cshtml) hiển thị danh sách sản phẩm bằng cách sử dụng Partial View.

```
<form asp-action="Login" asp-controller="Account">  
  <input asp-for="Username" />  
  <input asp-for="Password" />  
  <button type="submit">Đăng nhập</button>  
</form>
```



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU